

**UNIVERSIDAD MAYOR, REAL Y PONTIFICIA DE SAN FRANCISCO
XAVIER DE CHUQUISACA**



FACULTAD DE TECNOLOGÍA

INTELIGENCIA ARTIFICIAL II

Carrera: Ingeniería en Ciencias de la Computación

Título: Modelos Generativos Avanzados

Universitario: Monzón Bruno Antonio

Docente: Ing. Walter Pacheco Lora

**Sucre – Bolivia
2026**

ÍNDICE DE CONTENIDO

1. ¿Qué es un modelo generativo?	3
2. Modelos Generativos Clásicos vs Avanzados	3
2.1 Modelos Generativos Clásicos	3
2.2 Modelos Generativos Avanzados	3
3. Tipos de Modelos Generativos Avanzados	4
3.1 GAN – Generative Adversarial Network	4
3.2 VAE – Variational Autoencoder	5
3.3 Diffusion Models	6
4. Ejemplo Práctico: DCGAN	7
4.1 ¿Qué es una DCGAN?	7
4.2 Red Generadora (Generator)	7
4.3 Red Discriminadora (Discriminator)	8
4.4 Entrenamiento (función fit)	9
5. Aplicaciones Reales	10
6. Conclusión	11

1. ¿Qué es un Modelo Generativo?

Un modelo generativo es un tipo de inteligencia artificial diseñada para aprender los patrones subyacentes de un conjunto de datos y crear nuevos datos originales — texto, imágenes, audio — que se parecen a los de entrenamiento.

"A diferencia de los modelos discriminativos que clasifican datos, los modelos generativos inventan contenido nuevo."

¿Qué pueden generar?

Tipo de dato	Ejemplos de salida
Imágenes	Rostros, paisajes, arte, diseños
Texto	Artículos, código, diálogos, resúmenes
Audio	Música, voces sintéticas, efectos
Video	Escenas, animaciones, avatares

2. Modelos Generativos Clásicos vs Avanzados

La diferencia principal está en la complejidad, la autonomía y la fidelidad de los resultados que producen.

2.1 Modelos Generativos Clásicos

Son modelos estadísticos tradicionales que aprenden la distribución de un conjunto de datos para generar nuevos ejemplos similares.

- **Cómo funcionan:** Se basan en reglas matemáticas y probabilidades rígidas.
- **Resultado:** Limitados — pueden predecir la siguiente palabra en una frase sencilla, pero no mantienen coherencia en un párrafo largo.
- **Ejemplos:** Cadenas de Markov, Naive Bayes, Mezcla de Gaussianas (GMM).

"Analogía: Imita patrones superficiales pero no entiende nada. Como un loro que repite frases que escuchó."

2.2 Modelos Generativos Avanzados

Utilizan arquitecturas de Deep Learning para entender patrones extremadamente abstractos y profundos.

- **Cómo funcionan:** Usan redes neuronales masivas (Transformers, GANs, Diffusion) que no solo copian datos, sino que entienden el contexto, el estilo y la semántica.
- **Resultado:** Crean contenido que parece hecho por humanos, con creatividad y coherencia sorprendentes.
- **Ejemplos:** GPT-4 (texto), Midjourney y DALL·E (imágenes), Sora (video).

3. Tipos de Modelos Generativos Avanzados

Existen 3 grandes familias. Cada una tiene una estrategia diferente para aprender a generar datos nuevos.

3.1 GAN — Generative Adversarial Network (2014)

Autor: Ian Goodfellow et al.

Intuición

*"Un falsificador de arte que mejora porque tiene un detective que lo caza.
Ambos se vuelven mejores con el tiempo."*

¿Cómo funciona?

Dos redes neuronales compiten entre sí:

- **Generador:** Recibe ruido aleatorio y produce datos falsos (imágenes, audio).
- **Discriminador:** Recibe datos reales y falsos, y aprende a distinguirlos.

El generador mejora engañando al discriminador. El discriminador mejora detectando al generador. Se entrenan juntos hasta que el generador produce datos indistinguibles de los reales.

Ventajas

- Genera imágenes de muy alta calidad y nitidez.
- Rápido en la generación una vez entrenado.

Desventajas

- Entrenamiento inestable y difícil de balancear.
- Mode collapse: el generador puede rendirse y producir siempre la misma imagen.

3.2 VAE — Variational Autoencoder (2013)

Autores: Kingma & Welling

Intuición

*"Un compresor inteligente que no solo guarda datos comprimidos, sino
que aprende el idioma de los datos para inventar datos nuevos."*

¿Cómo funciona?

Dos redes que trabajan en equipo:

- **Encoder:** Comprime la imagen a un vector pequeño en el espacio latente.
- **Decoder:** Reconstruye la imagen desde ese vector.

La clave del VAE es que el espacio latente es probabilístico y continuo: en vez de guardar un punto exacto, guarda una distribución (media μ y varianza σ). Esto permite muestrear puntos nuevos y generar datos que nunca existieron.

Ventajas

- Entrenamiento muy estable.

- Ideal para detección de anomalías.

Desventajas

- Imágenes generadas más borrosas que las GANs.
- Menos fidelidad en detalles finos.

3.3 Diffusion Models (2020)

Autores: Ho et al. — paper 'Denoising Diffusion Probabilistic Models'

Intuición

"Aprendes a restaurar una foto dañada por el ruido. Una vez que dominas eso, puedes empezar desde ruido puro y restaurar una imagen que nunca existió."

¿Cómo funciona?

Dos procesos opuestos:

- **Proceso Forward (entrenamiento):** Toma una imagen real y le agrega ruido gaussiano paso a paso hasta convertirla en ruido aleatorio.
- **Proceso Reverse (generación):** La red aprende a revertir ese proceso — quitar el ruido paso a paso — hasta recuperar una imagen coherente.

4. Ejemplo Práctico: DCGAN

4.1 ¿Qué es una DCGAN?

DCGAN (Deep Convolutional GAN) es una GAN donde ambas redes (generador y discriminador) usan capas convolucionales en vez de capas densas simples.

"Las convoluciones entienden la estructura espacial de las imágenes: bordes, texturas, formas — exactamente lo que necesitamos para rostros."

4.2 Preparación de Datos (Dataset CelebA)

Se utiliza el dataset CelebA de Kaggle con 50,000 imágenes de rostros redimensionadas a 64×64 píxeles. A continuación se muestra la configuración del dataset:

```
class CelebADataset(Dataset):
    def __init__(self, data_dir, image_size=64):
        self.img_files = sorted([f for f in os.listdir(data_dir)
                                if f.endswith('.jpg')])[:50000]
        self.transform = transforms.Compose([
            transforms.Resize(image_size), # Redimensionar
            transforms.CenterCrop(image_size), # Recorte central
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5])
        ])
    ])
```

4.3 Red Generadora (Generator)

El Generador transforma un vector de ruido aleatorio (nz=100) en una imagen 64×64 usando capas ConvTranspose2d que aumentan la resolución progresivamente:

```
class Generator(nn.Module):
    def __init__(self, nz=100, ngf=64):
        super().__init__()
        self.main = nn.Sequential(
            # Ruido → 4x4 (512 filtros)
            nn.ConvTranspose2d(nz, ngf*8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf*8), nn.ReLU(True),
            # 4x4 → 8x8 (256 filtros)
            nn.ConvTranspose2d(ngf*8, ngf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*4), nn.ReLU(True),
            # 8x8 → 16x16 (128 filtros)
            nn.ConvTranspose2d(ngf*4, ngf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf*2), nn.ReLU(True),
            # 16x16 → 32x32 (64 filtros)
            nn.ConvTranspose2d(ngf*2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf), nn.ReLU(True),
            # 32x32 → 64x64 (3 canales RGB)
            nn.ConvTranspose2d(ngf, 3, 4, 2, 1, bias=False),
            nn.Tanh() # Salida en rango [-1, 1]
        )
    )
```

4.4 Red Discriminadora (Discriminator)

El Discriminador analiza imágenes 64×64 y devuelve una probabilidad entre 0 (falsa) y 1 (real) usando capas Conv2d que reducen la resolución:

```

class Discriminator(nn.Module):
    def __init__(self, ndf=64):
        super().__init__()
        self.main = nn.Sequential(
            # 64x64 → 32x32
            nn.Conv2d(3, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # 32x32 → 16x16
            nn.Conv2d(ndf, ndf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*2), nn.LeakyReLU(0.2, inplace=True),
            # 16x16 → 8x8
            nn.Conv2d(ndf*2, ndf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*4), nn.LeakyReLU(0.2, inplace=True),
            # 8x8 → 4x4
            nn.Conv2d(ndf*4, ndf*8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*8), nn.LeakyReLU(0.2, inplace=True),
            # 4x4 → 1x1 (puntuación final)
            nn.Conv2d(ndf*8, 1, 4, 1, 0, bias=False),
            nn.Sigmoid() # Probabilidad [0,1]
        )

```

4.5 Entrenamiento (función fit)

El entrenamiento alterna entre dos fases por cada batch:

- Fase 1 — Entrenar Discriminador: Se pasan imágenes reales (etiqueta 0.9) e imágenes falsas generadas (etiqueta 0), calculando el error combinado.
- Fase 2 — Entrenar Generador: El Generador produce imágenes y quiere que el Discriminador las clasifique como reales (etiqueta 1).

```

def fit(g, d, dataloader, epochs=30):
    g_optimizer = torch.optim.Adam(g.parameters(), lr=2e-4, betas=(0.5, 0.999))
    d_optimizer = torch.optim.Adam(d.parameters(), lr=2e-4, betas=(0.5, 0.999))
    crit = nn.BCELoss()

    for epoch in range(1, epochs+1):
        for X, _ in dataloader:
            X = X.to(device)
            bs = X.size(0)

            # Fase 1: Discriminador
            d.zero_grad()
            label_real = torch.ones(bs, 1).to(device) * 0.9 # Label
smoothing
            d_real_loss = crit(d(X), label_real)
            d_real_loss.backward()
            noise = torch.randn(bs, g.input_size).to(device)
            fake = g(noise)
            d_fake_loss = crit(d(fake.detach()),
torch.zeros(bs, 1).to(device))
            d_fake_loss.backward()
            d_optimizer.step()

            # Fase 2: Generador
            g.zero_grad()
            g_loss = crit(d(fake), torch.ones(bs, 1).to(device))
            g_loss.backward()
            g_optimizer.step()

```


5. Aplicaciones Reales

Arte y Entretenimiento

Herramienta	Modelo	¿Qué genera?
Midjourney	Diffusion	Imágenes por texto
Stable Diffusion	Diffusion + VAE	Imágenes open source
ElevenLabs	Transformer + Diffusion	Voces sintéticas
Sora (OpenAI)	Diffusion + Transformer	Video por texto

Ciencia y Medicina

- Generación de datos sintéticos para entrenar modelos con pocos datos reales.
- Diseño de moléculas: generar candidatos a medicamentos (AlphaFold, DiffDock).
- Imágenes médicas: aumentar datasets de radiografías o resonancias magnéticas.

Ciberseguridad

- Detección de anomalías: el VAE aprende lo normal y detecta lo inusual.
- Generación adversarial: testear sistemas de defensa con ataques sintéticos.

Industria

- Diseño generativo: piezas mecánicas optimizadas por IA.
- Simulación: datos sintéticos para entrenar vehículos autónomos.
- Privacidad: reemplazar datos reales con datos sintéticos equivalentes.

6. Conclusión

Los modelos generativos no solo analizan datos — los crean. Cada tipo tiene su propia lógica, pero todos comparten el mismo objetivo: aprender de ejemplos reales para producir algo nuevo que parezca igual de real.

Modelo	En palabras simples
GAN	Un falsificador y un detective que se entrenan mutuamente
VAE	Comprime una imagen a su esencia y la reconstruye
Transformer	Lee todo el texto a la vez y predice qué sigue
Diffusion	Parte de ruido puro y lo convierte en imagen paso a paso
DCGAN	Una GAN convolucional que generó rostros que nunca existieron

¿Por qué importa esto?

Las herramientas que probablemente ya usas hoy son combinaciones de estos mismos modelos:

- **ChatGPT / Claude:** Transformer que predice el texto más probable.
- **Midjourney / DALL·E:** Diffusion guiado por texto.
- **GitHub Copilot:** Transformer entrenado en millones de líneas de código.
- **Sora:** Diffusion aplicado a secuencias de imágenes.

"Antes, la IA solo clasificaba cosas. Ahora, la IA las crea — y cada año lo hace mejor. El reto ya no es técnico: es saber cuándo usarlos, para qué, y con qué responsabilidad."